Chapter 8
Linked Lists

How to read these notes: anything marked with the red NOT TESTED tag is included for completeness but is not required for this class. Everything else is fair game. Per the course directions: linked lists are 280 review, they are covered in lab, and you are expected to know them well for exams. Section 8.6 (`std::list<>` and `std::forward_list<>`) is not required.

8.1 Singly- and Doubly-Linked Lists

Recap: why not just use a vector

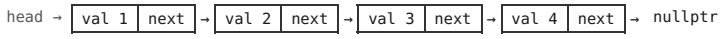
- `std::vector<>` stores data **contiguously** in memory.
- Contiguity gives constant time access to any element using **pointer arithmetic**.
- Contiguity also gives fast access due to **caching**: elements closer together in memory are faster to access sequentially.
- Downside: keeping elements contiguous makes some operations inefficient. Inserting or erasing anywhere other than the back requires shifting elements.

What a linked list is

- A **linked list** does not require elements to be stored contiguously.
- It is a sequence of **nodes**. Each node stores:
  - a data element, and
  - pointer(s) used to determine the relative ordering of nodes.

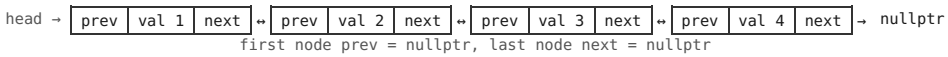
Singly-linked list

- Each node stores a pointer to the **next** node only.
- The last node's `next` points to `nullptr`.



Doubly-linked list

- Each node stores a pointer to both the **previous** and the **next** node.
- The last node's `next` and the first node's `prev` both point to `nullptr`.
- **Pro:** easy to traverse in both directions. **Con:** more memory because of the extra pointer.



Key consequence: no pointer arithmetic

- Because nodes are not contiguous, you cannot compute the address of an arbitrary node.
- To reach the *n*th element you must start at `head` and iterate *n* times: **`O(n)`**.

List advantages over a vector

- Memory is allocated **as needed**, so no excess capacity is wasted (a vector allocates extra capacity ahead of time).
- No **reallocation**. A vector must reallocate when size exceeds capacity, which costs time and invalidates pointers and iterators.
- **Pointers and iterators to a list element are never invalidated** until that element is destroyed, because a list element is never moved in memory.
- Adding or removing at the beginning or middle is asymptotically more efficient, since nothing after the modification point needs to be shifted.

List disadvantages

- Every node stores pointers in addition to its data, so a list usually uses **much more memory** than a properly sized or reserved vector holding the same data.
- Accessing an arbitrary element requires traversing everything before it.
- Lists cannot exploit memory contiguity and caching the way vectors can.
- Data access in a vector is often so much faster that **vectors frequently outperform lists in practice, even when lists have the asymptotic advantage**.

8.2 Linked List Node Insertion

The container class

- Node stores a value plus a `next` pointer (and `prev` if doubly-linked).
- The list keeps a **head** pointer to the first element, and an **optional tail** pointer to the last element.

```
template <typename T>
class LinkedList {
  struct Node {
    T data;
    Node* prev;
    Node* next;
    Node() : prev{nullptr}, next{nullptr} {}
    Node(T x) : data{x}, prev{nullptr}, next{nullptr} {}
  };

  Node* head;
  Node* tail;
public:
  LinkedList() {
    head = tail = nullptr;
  } // LinkedList()

  ~LinkedList() {              // destructor walks the list and deletes every node
    Node* temp;
    while (head != nullptr) {
      temp = head->next;
      delete head;
      head = temp;
    } // while
  } // ~LinkedList()
  ...
```

The four insertion cases

1. Insertion into an **empty** list.
2. Insertion at the **front**.
3. Insertion at the **back**.
4. Insertion in the **middle** (not beginning or end).

Case 1: empty list

- If the list is empty, `head` (and `tail`) are `nullptr`. Allocate the node and point both at it.

```
if (head == nullptr) {
  Node* new_node = new Node{val};
  head = new_node;
  tail = new_node;
} // if
```

Always check for nullptr before using `->`. Using `->` on a `nullptr` causes a segmentation fault.

Case 2: insert at the front

1. Allocate a new node.
2. Set the new node's `next` to `head`.
3. If doubly-linked, set the new node's `prev` to `nullptr` and `head->prev` to the new node.
4. Set `head` to the new node.

```
Node* new_node = new Node{val};   // allocate a new node, initialized to val
new_node->next = head;            // set next of this node to the current head
if (head != nullptr) {
  head->prev = new_node;          // list not empty: set prev of current head
} // if
else {
  tail = new_node;                // otherwise the new node is also the tail
} // else
head = new_node;                  // set head to the new node
```

Case 3: insert at the back

1. Allocate a new node.
2. If doubly-linked, set its `prev` to the last node (found via the tail pointer, or by iterating to the end if there is no tail pointer).
3. If there is a tail pointer, set it to the new node.

```
Node* new_node = new Node{val};   // allocate a new node, initialized to val
new_node->prev = tail;            // set prev to tail
if (tail != nullptr) {
  tail->next = new_node;          // list not empty: set next of tail to new node
} // if
else {
  head = new_node;                // otherwise set head to the new node
} // else
tail = new_node;                  // set tail to the new node
```

Case 4: insert in the middle (after a given `prev_node`)

- Allocate the node and update the connections of the two adjacent nodes (only the node before it if singly-linked).

```
Node* new_node = new Node{val};        // allocate a new node
new_node->prev = prev_node;            // update prev and next of the new node
new_node->next = prev_node->next;
prev_node->next = new_node;            // update next of prev_node
if (new_node->next != nullptr) {
  new_node->next->prev = new_node;     // update prev of node after insertion point
} // if
else {
  tail = new_node;                     // runs if prev_node was the last node
} // else
```

Insertion summary (doubly-linked)

Singly-linked is the same process, just without the `prev` steps.

Insert at beginning	Insert at end	Insert in middle, given `prev_node`
1. `new_node->next = head` 2. `new_node->prev = nullptr` 3. `head->prev = new_node` 4. `head = new_node`	1. `new_node->prev = tail` 2. `new_node->next = nullptr` 3. `tail->next = new_node` 4. `tail = new_node`	1. `new_node->prev = prev_node` 2. `new_node->next = prev_node->next` 3. `prev_node->next = new_node` 4. `new_node->next->prev = new_node`

EXAMPLE 8.1 **Insert into a sorted doubly-linked list**

Insert `val` so the list stays sorted. The class keeps `head` and `tail`; both are `nullptr` when empty. The `Node` constructor sets `prev` and `next` to `nullptr`. Node holds an `int32_t data`.

Duplicate rule chosen: insert a duplicate directly *after* existing duplicates. That way equality only has to be handled in the middle and end cases, never at the beginning.

1. List is empty (both pointers are `nullptr`): make the node both head and tail.

```
if (head == nullptr) {
  Node* new_node = new Node{val};
  head = new_node;  tail = new_node;
} // if
```

2. `val` is smaller than everything: since the list is sorted, just compare against the first element.

```
if (val < head->data) {
  Node* new_node = new Node{val};
  new_node->next = head;
  head->prev = new_node;
  head = new_node;
} // if
```

3. `val` is larger than everything: compare against the last element via `tail`.

```
if (val >= tail->data) {
  Node* new_node = new Node{val};
  new_node->prev = tail;
  tail->next = new_node;
  tail = new_node;
} // if
```

This case is easy only because a tail pointer exists. Without one you would iterate the whole list to reach the end.

4. `val` goes in the middle: find the node holding the largest value less than or equal to `val`, which is the node directly before the insertion point. Loop until the first node whose `next` is larger than `val`.

```
Node* prev_node = head;
while (prev_node->next && prev_node->next->data < val) {
  prev_node = prev_node->next;
} // while
```

No `nullptr` check is needed on `prev_node`: the loop only advances when `prev_node->next` is valid.

Full solution (member function). Cases 3 and 4 could be merged, which is required if there is no tail pointer, but then inserting at the end costs a full traversal.

```
void insert(int32_t val) {
  Node* new_node = new Node{val};
  if (head == nullptr) {            // case 1: empty
    head = new_node;  tail = new_node;
    return;
  } // if
  if (val < head->data) {           // case 2: front
    new_node->next = head;
    head->prev = new_node;
    head = new_node;
    return;
  } // if
  if (val >= tail->data) {          // case 3: back
    new_node->prev = tail;
    tail->next = new_node;
    tail = new_node;
    return;
  } // if
  Node* prev_node = head;           // case 4: middle
  while (prev_node->next && prev_node->next->data < val) {
    prev_node = prev_node->next;
  } // while
  new_node->prev = prev_node;
  new_node->next = prev_node->next;
  prev_node->next = new_node;
  new_node->next->prev = new_node;
} // insert()
```

8.3 Linked List Node Deletion

Deletion given an index

- Lists have no random access, so this is **`O(n)`** on average for both singly- and doubly-linked lists: you must iterate from the beginning to find the node.

Deletion given a pointer to the node

- **Doubly-linked: `O(1)`.** Just relink the two adjacent nodes.

```
node_to_delete->next->prev = node_to_delete->prev;
node_to_delete->prev->next = node_to_delete->next;
delete node_to_delete;
```

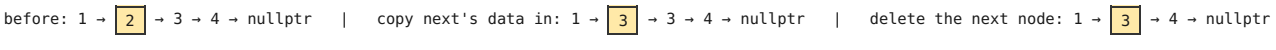
This code assumes deletion in the middle. Extra checks are needed if the node is at the beginning or end.

- **Singly-linked: still `O(n)`.** You must update the *previous* node's `next`, and there is no way to reach the previous node directly, so you must traverse.

Can a singly-linked list ever delete in Θ(1)?

- **Option A:** pass in a pointer to the node *directly before* the one to delete, i.e. `delete_node(Node* prev)`. This works but is counter-intuitive and, depending on the implementation, may not allow deleting the head node.
- **Option B (the copy trick):** if you must be given a pointer to the node itself:
  1. Overwrite the data of the node to delete with the next node's data.
  2. Delete the next node.

```
Node* next_node = node_to_delete->next;
node_to_delete->data = next_node->data;
node_to_delete->next = next_node->next;
delete next_node;
```



The copy trick does not always work. Three reasons:

1. It fails if the node to delete is the **last** node, since there is no next node to copy from and delete.
2. It assumes the data is **copyable**, which cannot be guaranteed.
3. It is **not safe**: you actually delete `node_to_delete->next`, not the node requested. The list looks right, but the memory addresses changed, so any pointers, iterators, or data depending on the address of `node_to_delete->next` are invalidated.

Conclusion: without a reference to the node directly before the target, worst-case deletion from a singly-linked list cannot be better than `O(n)`.

Deletion mirrors insertion: consider all edge cases and update surrounding nodes so the list stays valid.

EXAMPLE 8.2 **Delete a node at a given index from a singly-linked list**

0-indexed. The class only has a `head` pointer. Node stores `int32_t data` and `Node* next`.

Edge cases to consider:

1. List is empty: delete nothing.
2. Index is 0: update `head` before deleting.
3. Index is larger than the largest valid index: delete nothing.
4. If there were a tail pointer, deleting the last element would require updating it (does not apply here).

```
void delete_at_index(size_t index) {
  if (head == nullptr) {            // check for empty list
    return;
  } // if
  if (index == 0) {                 // check if index is 0
    Node* temp = head;
    head = temp->next;
    delete temp;
    return;
  } // if
  // find node before the one to delete; do not iterate off the end
  Node* prev_node = head;
  for (size_t i = 0; prev_node != nullptr && i < index - 1; ++i) {
    prev_node = prev_node->next;
  } // for i
  // make sure index is not larger than the largest index possible
  if (prev_node == nullptr || prev_node->next == nullptr) {
    return;
  } // if
  // delete the node and update the pointers
  Node* node_to_delete = prev_node->next;
  prev_node->next = node_to_delete->next;
  delete node_to_delete;
} // delete_at_index()
```

8.4 Reversing a Linked List

8.4.1 Naive solution

- Iterate through the list and copy each element to the **front** of a new list. Copying to the front reverses the order.
- When you hit `nullptr`, deallocate the old list and point `head` at the new list.

```
                                        original: 1 → 2 → 3 → 4 → nullptr
new list after each copy: [1]   [2 → 1]   [3 → 2 → 1]   [4 → 3 → 2 → 1]
```

```
void reverse_list(Node*& head) {
  Node* new_head = nullptr;
  while (head != nullptr) {
    Node* new_node = new Node{head->data};
    new_node->next = new_head;
    new_head = new_node;
    Node* old_node = head;
    head = head->next;
    delete old_node;              // deallocation happens during traversal here
  } // while
  head = new_head;
} // reverse_list()
```

- **Time `O(n)`, auxiliary space `O(n)`** (a full extra copy of the list).
- Wasteful: it duplicates memory and spends time constructing new nodes. If each node's data were large, duplicating every element could be very costly.

8.4.2 Optimized iterative solution (in-place)

1. Initialize three pointers: `curr`, `prev`, `next`. Set `curr = head`.
2. While `curr` is not `nullptr`:
   a. `next = curr->next`
   b. reverse the link: `curr->next = prev`
   c. move forward: `prev = curr`, then `curr = next`
3. After the loop (`curr` is `nullptr`), set `head = prev`.

```
                        step:   prev=null, curr=1, next=2  →  1's next points back to null
then prev=1, curr=2, next=3  →  2 points back to 1  ...  until curr=null, then head = prev = 4
                                result: 4 → 3 → 2 → 1 → nullptr
```

```
void reverse_list(Node*& head) {
  // initialize curr, prev, and next
  Node* curr = head;
  Node* prev = nullptr;
  Node* next = nullptr;
  while (curr != nullptr) {         // loop until curr is nullptr
    next = curr->next;              // update next to curr->next
    curr->next = prev;              // reverse curr's next pointer
    // move pointers forward by one; next is updated at the top of the next
    // iteration so that curr is guaranteed non-null before reading curr->next
    prev = curr;
    curr = next;
  } // while
  head = prev;                      // after the loop, set head to prev
} // reverse_list()
```

- **Time `O(n)`** (the whole list is traversed), **auxiliary space `O(1)`** (extra space does not depend on *n*).
- Done **in place**: no copy is made and no old list needs to be deallocated afterward.
- The same approach reverses a **doubly**-linked list in `O(1)` auxiliary space; the only difference is that two directional pointers are adjusted per step instead of one.

8.4.3 Optimized recursive solution

1. Initialize `curr` to the head of the list.

2. Base cases:
   a. if `curr` is `nullptr`, return;
   b. if `curr->next` is `nullptr`, this is the last node, so make it the new head and return.
3. Recurse on `curr->next`.
4. Set `curr->next->next = curr`.
5. Set `curr->next = nullptr`.

Walkthrough on 1 → 2 → 3 → 4: recursion descends to node 4, where the base case fires and 4 becomes the new head. The recursion then unrolls back through 3, 2, and 1, and at each node steps 4 and 5 flip that node's link.

```
void helper(Node*& head, Node*& curr) {
  if (curr == nullptr) {
    return;
  } // if
  if (curr->next == nullptr) {
    head = curr;
    return;
  } // if
  helper(head, curr->next);
  curr->next->next = curr;
  curr->next = nullptr;
} // helper()

void reverse_list(Node *&head) {
  Node* curr = head;
  helper(head, curr);
} // reverse_list()
```

Remark: pointer by reference (`Node*& head`)

- Passing a pointer **by reference** lets the function change where the original pointer points.
- Passing by value gives the function a **local copy** of the pointer; reassigning it does not affect the caller's pointer.

```
void foo(Node* head)  { head = nullptr; }   // by value
void bar(Node*& head) { head = nullptr; }   // by reference

int main() {
  Node* head = new Node{281};
  foo(head);
  std::cout << head << std::endl;  // prints 0xd20c20 (arbitrary), NOT changed
  bar(head);
  std::cout << head << std::endl;  // prints 0x0, changed
} // main()
```

Summary: if you pass a pointer into `f()` by reference, any change to the pointer inside `f()` is also seen by the caller.

8.5 Techniques for Solving Linked List Problems

The two-pointer technique. Move two pointers through the list, either (a) at a **fixed distance apart**, or (b) with one moving **faster** than the other. Most classic list problems reduce to one of these two setups.

EXAMPLE 8.3 **Find the *k*th to last element (singly-linked)**

List is non-empty and *k* is valid. Return the `data` value.

Idea: the *k*th to last element is exactly *k* away from the end, so use two pointers a distance of *k* nodes apart. When the front pointer hits the end, the back pointer must be on the *k*th to last node.

1. Point `fast` and `slow` at `head`.
2. Move `fast` forward *k* positions.
3. Advance both together until `fast` is `nullptr`.
4. Return `slow->data`.

```
int32_t kth_to_last(Node* head, int32_t k) {
  Node* fast = head;
  Node* slow = head;
  // move the fast pointer forward by k positions
  // k is always valid here, otherwise you must check for nullptr
  for (int32_t i = 0; i < k; ++i) {
    fast = fast->next;
  } // for i
  // move both forward until fast reaches the end
  while (fast != nullptr) {
    fast = fast->next;
    slow = slow->next;
  } // while
  return slow->data;
} // kth_to_last()
```

EXAMPLE 8.4 **Find the middle node (singly-linked)**

If there are two middle nodes, return the **second** one.

Idea: make one pointer move twice as fast as the other. Start both at the head, advance `fast` by 2 and `slow` by 1 each iteration. When `fast` reaches the end, `slow` is at the middle.

```
int32_t find_middle_node(Node* head) {
  Node* fast = head;
  Node* slow = head;
  // only fast needs validity checks, since fast hits the end of the list first
  while (fast != nullptr && fast->next != nullptr) {
    slow = slow->next;
    fast = fast->next->next;
  } // while
  return slow->data;
} // find_middle_node()
```

EXAMPLE 8.5 **Detect a cycle: Floyd's Cycle-Finding Algorithm**

A cycle exists when some node points back to a previous node in the list.

Building up the idea:

- If you wanted to detect a cycle of **known length 6**, you could keep two pointers 6 nodes apart and advance them at the same rate. In a cycle of length 6, the node 6 steps ahead of a pointer is itself, so the pointers would eventually coincide.
- The same trick works for length 7 with pointers 7 apart, and so on. But we do not know the cycle length in advance, so a fixed distance is not enough.
- **Fix:** move one pointer at twice the speed of the other. Then the gap between them grows by 1 each iteration, so the gap eventually equals the cycle length no matter what that length is. If the two pointers ever land on the same node, a cycle exists.

What if the gap already exceeds the cycle length when `slow` enters the cycle?

- Still fine. Multiples of the cycle size also work. In a cycle of length 20, the node 40 (or 60, or 80) steps away is the same node.
- No matter how far the cycle is from the head, both pointers eventually end up inside the cycle, and the first multiple of the cycle length detects it.

```
bool has_cycle(Node* head) {
  // two pointers, one faster; if fast catches slow, there is a cycle
  Node* fast = head;
  Node* slow = head;
  while (fast && fast->next) {
    slow = slow->next;
    fast = fast->next->next;
    if (fast == slow) {
      return true;
    } // if
  } // while
  return false;
} // has_cycle()
```

EXAMPLE 8.6 **Partition a list around a value `p`**

All nodes with value $<$ `p` must come before all nodes with value $\geq$ `p`, and relative order must be preserved inside each partition.

`input (p = 3): 2 → 8 → 1 → 3 → 7 → 0  ⇒  output: 2 → 1 → 0 → 8 → 3 → 7`

Key insight: the answer is two sublists joined together, one holding values $<$ `p` and one holding values $\geq$ `p`. Iterate the original list once and move each node into one of the two sublists, then link them.

- `before_head` / `before_ptr` track the $<$ `p` sublist; `after_head` / `after_ptr` track the $\geq$ `p` sublist.
- The `*_ptr` pointers mark the insertion position (the current tail) of each sublist.
- A **dummy node** at the head of each sublist removes the empty-list special case, which is why the return value uses `.next`.

```
Node* partition_list(Node* head, int32_t p) {
  // dummy nodes at the head of each sublist make the implementation easier
  Node before_head{0}, *before_ptr = &before_head;
  Node after_head{0}, *after_ptr = &after_head;
  while (head != nullptr) {
    if (head->val < p) {
      before_ptr->next = head;
      before_ptr = before_ptr->next;
    } // if
    else {
      after_ptr->next = head;
      after_ptr = after_ptr->next;
    } // else
    head = head->next;
  } // while
  // combine the two lists
  after_ptr->next = nullptr;
  before_ptr->next = after_head.next;
  return before_head.next;
} // partition_list()
```

Time `O(n)` (one pass over all nodes). **Auxiliary space `O(1)`**: nodes are reorganized, not duplicated, so the extra memory is a constant.

EXAMPLE 8.7 **Find the intersection node of two singly-linked lists**

Return the node where the two lists intersect, or `nullptr`. Assume no cycles.

Warm-up: if both lists have the **same number of nodes before** the intersection, just walk both pointers in tandem. When they land on the same node, that is the intersection. **Solution 1: precompute the lengths.** Compute both lengths, advance the pointer in the longer list by the difference to line them up, then walk in tandem.

```
int32_t get_list_length(Node* head) {
  int32_t length = 0;
  while (head != nullptr) { ++length; head = head->next; }
  return length;
} // get_list_length()

Node* get_list_intersection(Node* head1, Node* head2) {
  int32_t len1 = get_list_length(head1);
  int32_t len2 = get_list_length(head2);
  // advance the pointer in the longer list to line up with the shorter one
  if (len1 < len2) {
    int32_t diff = len2 - len1;
    for (int32_t i = 0; i < diff; ++i) { head2 = head2->next; }
  } // if
  else if (len2 < len1) {
    int32_t diff = len1 - len2;
    for (int32_t j = 0; j < diff; ++j) { head1 = head1->next; }
  } // else if

  while (head1 != nullptr && head2 != nullptr && head1 != head2) {
    head1 = head1->next;
    head2 = head2->next;
    if (head1 == head2) { return head1; }
  } // while
  // ternary handles the case where head1 and head2 start at the same node
  return head1 == head2 ? head1 : nullptr;
} // get_list_intersection()
```

Solution 2: no length precomputation. Walk both pointers in tandem regardless of alignment. When a pointer reaches the end, **restart it at the head of the opposite list**. Both pointers then travel the same total distance and must meet at the intersection.

Why it works. Let x = length of list 1 before the intersection, y = length of list 2 before the intersection, z = length of the shared tail.

- Pointer starting on the shorter head travels x, then z to the end, then y in the other list.
- Pointer starting on the longer head travels y, then z, then x.
- Both visit exactly $\boldsymbol{x + y + z}$ nodes, so they arrive at the intersection at the same time.

```
Node* get_list_intersection(Node* head1, Node* head2) {
  Node *p1 = head1, *p2 = head2;
  if (p1 == nullptr || p2 == nullptr) { return nullptr; }

  while (p1 != nullptr && p2 != nullptr && p1 != p2) {
    p1 = p1->next;
    p2 = p2->next;
    if (p1 == p2) { return p1; }        // both met at the same node
    if (p1 == nullptr) { p1 = head2; }  // restart p1 at head2
    if (p2 == nullptr) { p2 = head1; }  // restart p2 at head1
  } // while()
  return p1;
} // get_list_intersection()
```

Concise version. The ternaries reset the pointers at the end, and the loop condition handles the no-intersection case (both eventually become `nullptr`).

```
Node* get_list_intersection(Node* head1, Node* head2) {
  Node *p1 = head1, *p2 = head2;
  while (p1 != p2) {
    p1 = p1 ? p1->next : head2;
    p2 = p2 ? p2->next : head1;
  } // while()
  return p1;
} // get_list_intersection()
```

Time `O(n)` where n is the total number of nodes in the combined lists. With no intersection the pointers may visit more than n nodes but never more than $2n$, since the algorithm stops once both hit `nullptr` on the second traversal, which is still `O(n)`. **Auxiliary space `O(1)`.**

EXAMPLE 8.8 **Add two numbers stored as linked lists**

Digits are stored in **reverse order**: least significant digit at the head, most significant at the tail. Each node holds one digit (`int8_t digit`), no leading zeros, both lists non-empty. Return the sum in the same format.

`head1: 1 → 8 → 2 (= 281)  head2: 0 → 7 → 3 (= 370)  ⇒  result: 1 → 5 → 6 (= 651)`

Idea: this is grade-school addition. Add digits from least significant to most significant and carry anything over 9 to the next column. Because the digits are already reversed, a single front-to-back pass over both lists works, plus a `carry` variable.

- 1 + 0 = 1, append 1, carry 0
- 8 + 7 = 15, append 5, carry 1
- 1 + 2 + 3 = 6, append 6, carry 0

Edge cases:

1. **One list is longer than the other**: do not iterate off the end of the shorter list.
2. **There is a leftover carry at the end** (for example 1 + 999): append it as an extra node.

```
Node* add_two_lists(Node* head1, Node* head2) {
  // dummy node again; pre_head.next is the head of the list to return
  Node pre_head{0}, *result = &pre_head;
  Node *iter1 = head1, *iter2 = head2;
  int8_t carry = 0;
  // keep going as long as either list still has nodes
  while (iter1 != nullptr || iter2 != nullptr) {
    int8_t sum = (iter1 ? iter1->val : 0) + (iter2 ? iter2->val : 0) + carry;
    result->next = new Node{sum % 10};   // next digit is sum mod 10
    result = result->next;
    carry = sum / 10;                    // carry is 0 or 1 here
    if (iter1 != nullptr) { iter1 = iter1->next; }
    if (iter2 != nullptr) { iter2 = iter2->next; }
  } // while
  if (carry != 0) {                      // leftover carry becomes a final node
    result->next = new Node{carry};
  } // if
  return pre_head.next;
} // add_two_lists()
```

Time `O(max(m, n))` for lists of length m and n, since the loop runs as long as the longer list. **Auxiliary space `O(max(m, n))`**, since the result has at least max(m, n) and at most max(m, n) + 1 digits.

Remark: what if the most significant digit were at the head instead? A singly-linked list cannot be walked backwards, so you cannot iterate in the needed order directly. Two fixes:

1. Reverse the lists first using the algorithm from section 8.4, then run the same process.
2. Use a last-in, first-out container such as a `std::stack<>` to access the digits in the right order (chapter 9).

8.6 The STL List Containers NOT TESTED

NOT REQUIRED FOR THIS CLASS

Course directions: you do **not** need to know how to use `std::list<>` or `std::forward_list<>`. Included here as reference only.

8.6.1 `std::list<>` (doubly-linked list, `#include <list>`)

Constructors

- `list<T>()` default, empty, size 0
- `list<T>(const list<T>& other)` copy constructor
- `list<T>(size_t sz)` sz value-initialized elements
- `list<T>(size_t sz, T& val)` sz copies of val
- `list<T>(InputIterator begin_iter, InputIterator end_iter)` range [begin, end), inclusive begin and exclusive end
- `list<T>(std::initializer_list<T> init)`

Front and back insertion / removal

- `push_back(const T& val)`, `push_front(const T& val)`
- `emplace_back(Args&&... args)`, `emplace_front(Args&&... args)`: construct in place; return a reference since C++17
- `pop_back()`, `pop_front()`: references and iterators to the erased element are invalidated; undefined behavior on an empty list
- `resize(size_t sz, const T& val)`: second argument optional (new elements are value-initialized if omitted); if the current size is larger than sz, the list is cut down to the first sz elements

`.insert()` and `.erase()` (behave like the vector versions)

- `insert(const_iterator pos, const T& val)` inserts directly before pos, returns iterator to the new element
- `insert(pos, size_t n, const T& val)` n copies, returns iterator to the first added
- `insert(pos, InputIterator first, InputIterator last)` range [first, last), returns iterator to first added
- `insert(pos, std::initializer_list<T> init)` returns iterator to first added
- `emplace(const_iterator pos, Args&&... args)` constructs in place before pos, returns iterator to it
- `erase(iterator pos)` returns iterator to the element after the erased one
- `erase(iterator first, iterator last)` erases [first, last), returns iterator after the last erased

`.splice()`: transfers nodes to another list or another position in the same list. It only repoints internal pointers, so **no iterators or references are invalidated** and splicing a single item takes **constant time**.

- `splice(const_iterator pos, list& other)` moves all of other before pos; other becomes empty; undefined behavior if other is `*this`
- `splice(pos, other, const_iterator it)` moves the single element at it
- `splice(pos, other, first, last)` moves range [first, last); undefined behavior if pos is inside [first, last)

Other members: `.front()`, `.back()`, `.empty()`, `.size()`, `.clear()`, `.begin()` / `.end()` (bidirectional iterators), `.cbegin()` / `.cend()` (constant), `.rbegin()` / `.rend()` (reverse), `.crbegin()` / `.crend()` (constant reverse), and `.sort()` (ascending, or with an optional comparator).

Lists have their own `.sort()` because the generic `std::sort()` from `<algorithm>` cannot be used on a list.

```
std::list<int32_t> lst1;                // empty
std::list<int32_t> lst2 = {1, 2, 3};
lst2.push_front(0);                     // {0,1,2,3}
lst2.push_back(4);                      // {0,1,2,3,4}
std::list<int32_t> lst3{lst2};          // copy: {0,1,2,3,4}
lst3.pop_front();                       // {1,2,3,4}
lst3.pop_back();                        // {1,2,3}
lst3.resize(5);                         // {1,2,3,0,0}
lst3.sort();                            // {0,0,1,2,3}
```

8.6.2 `std::forward_list<>` (singly-linked list, `#include <forward_list>`)

- Preferred over `std::list<>` when bidirectional iteration is not needed, since it stores no `prev` pointer and saves memory.
- More limited: **no** `.push_back()`, **no** `.pop_back()`, **no** reverse iterators, and **no** `.size()` (omitted for efficiency).
- Optimized for containers that are typically empty or very small.

Members

- Constructors mirror `std::list<>`: default, copy, `(sz)`, `(sz, val)`, iterator range [begin, end), initializer list
- `clear()`, `empty()`
- `insert_after(iterator pos, const T& val)` and the `const_iterator` overload: insert after pos, return iterator to the inserted element
- `insert_after(pos, size_t n, const T& val)` returns iterator to the last element inserted
- `insert_after(pos, InputIterator first, InputIterator last)` inserts range after pos, returns iterator to the last inserted
- `insert_after(pos, std::initializer_list<T> init)` returns iterator to the last inserted, or pos if the list is empty
- `emplace_after(const_iterator pos, Args&&... args)` constructs in place after pos
- `erase_after(const_iterator pos)` erases the element after pos, returns iterator to the one following it
- `erase_after(const_iterator first, const_iterator last)` erases everything after first up to last, returns iterator after the last erased
- `push_front(const T& val)`, `emplace_front(Args&&... args)` (returns a reference since C++17), `pop_front()` (undefined behavior if empty)

8.7 Summary of List Complexities

A head pointer is required; a tail pointer is optional. Both variations are shown. Average and worst case are identical for every operation listed.

Operation	Singly-linked	Doubly-linked	Why / STL equivalent
Finding an element	`O(n)`	`O(n)`	Worst case: the element is last or absent, so all *n* nodes are checked. Average: about *n*/2 elements, still Θ(n) after dropping the 1/2. STL: `std::find()` from `<algorithm>` (chapter 11).
Accessing first element	`O(1)`	`O(1)`	The head pointer points straight at it. STL: `.front()` on both list and forward list.
Accessing last element	`O(1)` with tail `O(n)` without	`O(1)` with tail `O(n)` without	With a tail pointer just read it, otherwise iterate all *n* nodes. STL list: `.back()`. Forward lists have no `.back()`, so you iterate to `.end()`.
Accessing arbitrary element	`O(n)`	`O(n)`	Not contiguous, so no pointer arithmetic. Worst case all *n* nodes, average *n*/2, both Θ(n).
Inserting at front	`O(1)`	`O(1)`	Just repoint a few pointers; head gives direct access. STL: `.push_front()`, `.emplace_front()`.
Inserting at back	`O(1)` with tail `O(n)` without	`O(1)` with tail `O(n)` without	Without a tail pointer you must traverse first. If you are handed the node to insert after, it is Θ(1). STL list: `.push_back()`, `.emplace_back()`. Forward list: use `.insert_after()` / `.emplace_after()` on the last element.
Inserting at arbitrary index	`O(n)`	`O(n)`	You must iterate to the index first. Given a **pointer** to the node to insert after, it is Θ(1). STL: `.insert()` / `.emplace()`, or `.insert_after()` / `.emplace_after()`; these take constant time but you must supply an iterator, which itself may cost a traversal.
Erasing at front	`O(1)`	`O(1)`	Head gives direct access, just shift pointers. STL: `.pop_front()` on both.
Erasing at back	`O(n)` (even with tail)	`O(1)` with tail `O(n)` without	Singly-linked: you must set the `next` of the node *before* the last one, which you cannot reach in constant time, even with a tail pointer. Doubly-linked: the last node's `prev` gets you there. STL list: `.pop_back()`. Forward list has no `.pop_back()`, so find the element before the back and call `.erase_after()`.
Erasing at arbitrary index	`O(n)`	`O(n)`	No pointer arithmetic, so you must iterate to the position first.
Erasing given its pointer	`O(n)`	`O(1)`	You must update the previous node's `next` (and the next node's `prev` if doubly-linked). A doubly-linked list reaches both neighbors in constant time; a singly-linked list has no `prev`, so it must traverse. STL: `.erase()` for lists, `.erase_after()` for forward lists.
Checking if empty	`O(1)`	`O(1)`	Check whether `head == nullptr`. STL: `.empty()`.

Facts worth memorizing

- Erasing at the **back of a singly-linked list is always Θ(n)**, tail pointer or not.
- Erasing **given a pointer** is Θ(1) only for doubly-linked lists.
- A tail pointer helps **insertion** at the back, never **deletion** at the back of a singly-linked list.
- Anything phrased as "given an index" costs a traversal; anything phrased as "given a node pointer or iterator" usually does not.
- Determining whether a **doubly**-linked list is circular is Θ(1): check if the head's `prev` is the last node. For a singly-linked list it is Θ(n).
- Splicing an entire list into another doubly-linked list at a given iterator is **Θ(1)**, independent of both lengths.
- Merging two sorted lists of size *n* is **Θ(n)** with the two-pointer merge.
- Reversing is **Θ(n)** in every variation; a tail pointer does not improve it.